Established – 1961 **Subject: _____**

SEVA SADAN'S

R. K. TALREJA COLLEGE

OF

ARTS, SCIENCE & COMMERCE

ULHASNAGAR – 421 003

CERTIFICATE

THIS IS TO CERTIFY THAT MR./MS._____

_____

OF F.Y./ S.Y./ T.Y.B. COM / B. SC. CLASS ROLL NO. _____ HAS SATISFACTORILY COMPLETED THE PRACTICAL IN COMPUTER SCIENCE /MATHEMATICS / INFORMATION TECHNOLOGY IN THE YEAR 20 – 20.

PROF. INCHARGE EXAMINER HEAD OF THE DEPT

TABLE OF CONTENTS

Sr. No.	Chapter Title	Page No.
1	Introduction	5
2	Problem Definition	6
3	Objectives of the Project	7
4	Scope of the Project	8
5	Requirement Specification	10
6	System Design	11
7	Database Design	12
8	UML Diagrams	15
9	SQL Implementation	22
10	System Testing and Result	26
11	Security, Backup and Recovery	28
12	Future Scope and Conclusion	30
13	References	32
14	Glossary	33

CHAPTER 1

INTRODUCTION

The Insurance Policy Management System (IPMS) is a database-oriented application developed to manage insurance-related information in an organised, secure, and efficient manner. Insurance companies routinely deal with large volumes of sensitive data encompassing customer details, policy records, premium payments, and agent information. Managing such data manually or through unstructured digital systems frequently results in data redundancy, inconsistency, and significant operational inefficiency.

In the modern digital era, Relational Database Management Systems (RDBMS) have become indispensable tools for maintaining accurate, consistent, and readily accessible data. RDBMS platforms provide a structured mechanism to store, retrieve, update, and delete data while enforcing data integrity through various constraints.

This project employs MySQL, a widely adopted open-source RDBMS, to implement a structured database that models real-world insurance operations. MySQL is chosen for this project owing to its reliability, ease of use, open-source availability, strong community support, and compatibility with various operating systems. The system automates critical activities such as customer registration, policy assignment, premium payment tracking, and report generation.

This project is developed as part of the SY B.Sc. IT curriculum at R. K. Talreja College, University of Mumbai. It aims to provide practical exposure to database design, normalisation, SQL operations, and data security concepts as outlined in the RDBMS syllabus.

CHAPTER 2

PROBLEM DEFINITION

Traditional insurance management systems rely heavily on manual record-keeping or loosely structured digital files such as spreadsheets and text documents. Such approaches introduce several significant problems that adversely affect efficiency, accuracy, and reliability.

The core challenges identified in existing systems are as follows:

- Manual paperwork is time-consuming and highly prone to human error, leading to incorrect policy records and payment details.
- Physical records are vulnerable to loss, damage, or misplacement due to unforeseen circumstances such as fire or flooding.
- Lack of a centralised storage system causes data duplication, where the same customer or policy information is recorded multiple times.
- Searching and updating records manually is slow and inefficient, particularly when dealing with thousands of customers and policies.
- No proper security or access control mechanisms exist, resulting in unauthorised access to confidential customer information.
- There is a high risk of inconsistent or incorrect data updates when multiple staff members modify records independently.
- Generating accurate reports and statistical summaries is difficult without a structured query mechanism.

These challenges collectively reduce organisational productivity and negatively impact the quality of customer service. The Insurance Policy Management System (IPMS) addresses these issues comprehensively by providing a centralised, database-driven solution developed using MySQL. It ensures data integrity, security, and rapid retrieval of information through structured SQL queries.

CHAPTER 3

OBJECTIVES OF THE PROJECT

The objectives of the Insurance Policy Management System are clearly defined to align with the academic scope and the practical requirements of an insurance organisation. The primary objectives of this project are as follows:

- To design a structured and normalised relational database using MySQL that accurately represents the data requirements of an insurance management system.
- To store and manage insurance-related data, including customer profiles, insurance policies, policy enrolments, premium payments, and agent details, in an efficient and organised manner.
- To retrieve meaningful and actionable information using structured SQL queries such as SELECT, JOIN, and aggregate functions.
- To enforce data integrity using constraints such as PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, and CHECK to ensure consistency and accuracy of stored data.
- To implement Data Control Language (DCL) commands for managing user access and enforcing role-based security.
- To utilise Transaction Control Language (TCL) commands to ensure reliable and atomic database transactions.
- To create SQL views for simplified and secure access to sensitive data without exposing underlying table structures.
- To perform database normalisation up to the Third Normal Form (3NF) to eliminate redundancy and prevent update anomalies.
- To gain hands-on practical experience with MySQL Workbench and SQL operations as part of the SY B.Sc. IT curriculum.

CHAPTER 4

SCOPE OF THE PROJECT

The scope of the Insurance Policy Management System defines its functional coverage, intended user base, and academic limitations. It provides a clear understanding of what the system accomplishes and the boundaries within which it operates.

4.1 Functional Scope

The system covers the following key functional areas:

- Customer registration and profile management, including storage of personal details such as name, contact information, address, and date of birth.
- Insurance policy creation and classification, enabling the definition of various policy types along with their associated sum assured, premium amount, and duration.
- Policy enrolment and tracking, allowing customers to be enrolled in one or more insurance policies and monitoring the enrolment status.
- Premium payment recording, enabling systematic recording of payment dates and amounts against each policy enrolment.
- Agent information and commission tracking, maintaining records of insurance agents along with their contact details and applicable commission rates.

4.2 User Scope

The intended users of the system are:

- Insurance administrators who have full access to all database operations.
- Office staff who can insert and update customer and payment records.
- Insurance agents with limited read access to policy and enrolment information.

4.3 Academic Scope

This project focuses exclusively on backend database implementation using MySQL. The development of a graphical user interface (GUI), real-time payment gateway integration, and multi-user concurrency management are beyond the scope of this academic project. The project serves as a demonstration of RDBMS concepts studied in Semester IV of the SY B.Sc. IT programme.

CHAPTER 5

REQUIREMENT SPECIFICATION

This chapter outlines the hardware and software requirements necessary for the development and execution of the Insurance Policy Management System.

5.1 Hardware Requirements

Table 5.1: Hardware Requirements

Hardware Component	Minimum Specification
Processor	Intel Core i3 or equivalent (1 GHz or above)
RAM	4 GB (8 GB recommended)
Storage	50 GB available hard disk space
Display	1024 x 768 resolution or higher

5.2 Software Requirements

Table 5.2: Software Requirements

Software	Purpose
MySQL Server 8.0	Core database management system
MySQL Workbench 8.0	Graphical interface for query execution and schema design
Windows 10 / Ubuntu 20.04	Operating system platform
SQL	Query language for database operations

CHAPTER 6

SYSTEM DESIGN

6.1 System Architecture

The Insurance Policy Management System follows a single-tier architecture where the user interacts directly with the MySQL database through the MySQL Workbench interface. The architecture consists of three logical layers:

- User Interface Layer: MySQL Workbench serves as the front-end tool through which users write and execute SQL queries.
- Query Processing Layer: The MySQL Server receives and parses SQL queries, validates them against the database schema, and executes them using the query optimiser.
- Data Storage Layer: The physical data is stored in structured tables on the disk, managed by the InnoDB storage engine, which supports ACID-compliant transactions.

When a user submits a query through MySQL Workbench, the query is transmitted to the MySQL Server. The server processes the query, interacts with the relevant tables in the database, and returns the result set to the user. This architecture ensures efficient query execution and reliable data management.

CHAPTER 7

DATABASE DESIGN

7.1 Entity Description

The database consists of five entities, each representing a key aspect of the insurance management domain:

- Customers: Stores personal information about each individual who holds or intends to hold an insurance policy, including name, email, phone number, address, and date of birth.
- Insurance_Policies: Defines the various types of insurance policies offered by the company, including policy type, sum assured, premium amount, and duration.
- Policy_Enrollment: Establishes the relationship between customers and insurance policies, recording enrolment details including the start date and current status.
- Payments: Records premium payment transactions made against each policy enrolment, storing the payment date and amount paid.
- Agents: Stores information about insurance agents, including their contact details and commission rates.

7.2 Table Structure

The following tables describe the attributes and data types of each entity in the database.

Table 7.1: Customers Table

Attribute	Data Type	Constraint
cust_id	INT	PRIMARY KEY, AUTO_INCREMENT
first_name	VARCHAR(50)	NOT NULL
last_name	VARCHAR(50)	NOT NULL
email	VARCHAR(100)	UNIQUE, NOT NULL
phone_no	VARCHAR(15)	NOT NULL

Attribute	Data Type	Constraint
address	TEXT	
dob	DATE	NOT NULL

Table 7.2: Insurance_Policies Table

Attribute	Data Type	Constraint
policy_id	INT	PRIMARY KEY, AUTO_INCREMENT
policy_type	VARCHAR(100)	NOT NULL
sum_assured	DECIMAL(15,2)	NOT NULL
premium_amount	DECIMAL(10,2)	NOT NULL
duration	INT	NOT NULL

Table 7.3: Policy_Enrollment Table

Attribute	Data Type	Constraint
enroll_id	INT	PRIMARY KEY, AUTO_INCREMENT
cust_id	INT	FOREIGN KEY (Customers)
policy_id	INT	FOREIGN KEY (Insurance_Policies)
start_date	DATE	NOT NULL
status	VARCHAR(20)	DEFAULT 'Active'

Table 7.4: Payments Table

Attribute	Data Type	Constraint
payment_id	INT	PRIMARY KEY, AUTO_INCREMENT
enroll_id	INT	FOREIGN KEY (Policy_Enrollment)
payment_date	DATE	NOT NULL
amount	DECIMAL(10,2)	NOT NULL

Table 7.5: Agents Table

Attribute	Data Type	Constraint
agent_id	INT	PRIMARY KEY, AUTO_INCREMENT
agent_name	VARCHAR(100)	NOT NULL
contact_no	VARCHAR(15)	NOT NULL
commission_rate	DECIMAL(5,2)	NOT NULL

7.3 Constraints Used

- PRIMARY KEY: Applied to all identity columns to uniquely identify each record within its respective table.
- FOREIGN KEY: Applied in Policy_Enrollment and Payments tables to enforce referential integrity between related tables.
- NOT NULL: Applied to all critical fields to ensure that mandatory data is always provided during record insertion.
- UNIQUE: Applied to the email column in the Customers table to prevent duplicate registrations.
- DEFAULT: Applied to the status column in Policy_Enrollment to automatically assign 'Active' when no value is specified.

7.4 Normalisation

The database is normalised up to the Third Normal Form (3NF) to eliminate data redundancy and prevent update, insertion, and deletion anomalies.

- First Normal Form (1NF): All tables contain atomic values with no repeating groups.
- Second Normal Form (2NF): All non-key attributes are fully functionally dependent on the entire primary key. Partial dependencies have been removed.
- Third Normal Form (3NF): All non-key attributes are directly dependent on the primary key. Transitive dependencies have been eliminated.

CHAPTER 8

UML DIAGRAMS

8.1 Entity Relationship (ER) Diagram

The Entity Relationship (ER) Diagram visually represents the entities in the database and the relationships between them. A Customer can enrol in multiple Insurance Policies through the Policy_Enrollment entity. Each enrolment can have multiple Payments associated with it. The key cardinalities are one-to-many between Customers and Policy_Enrollment, one-to-many between Insurance_Policies and Policy_Enrollment, and one-to-many between Policy_Enrollment and Payments.

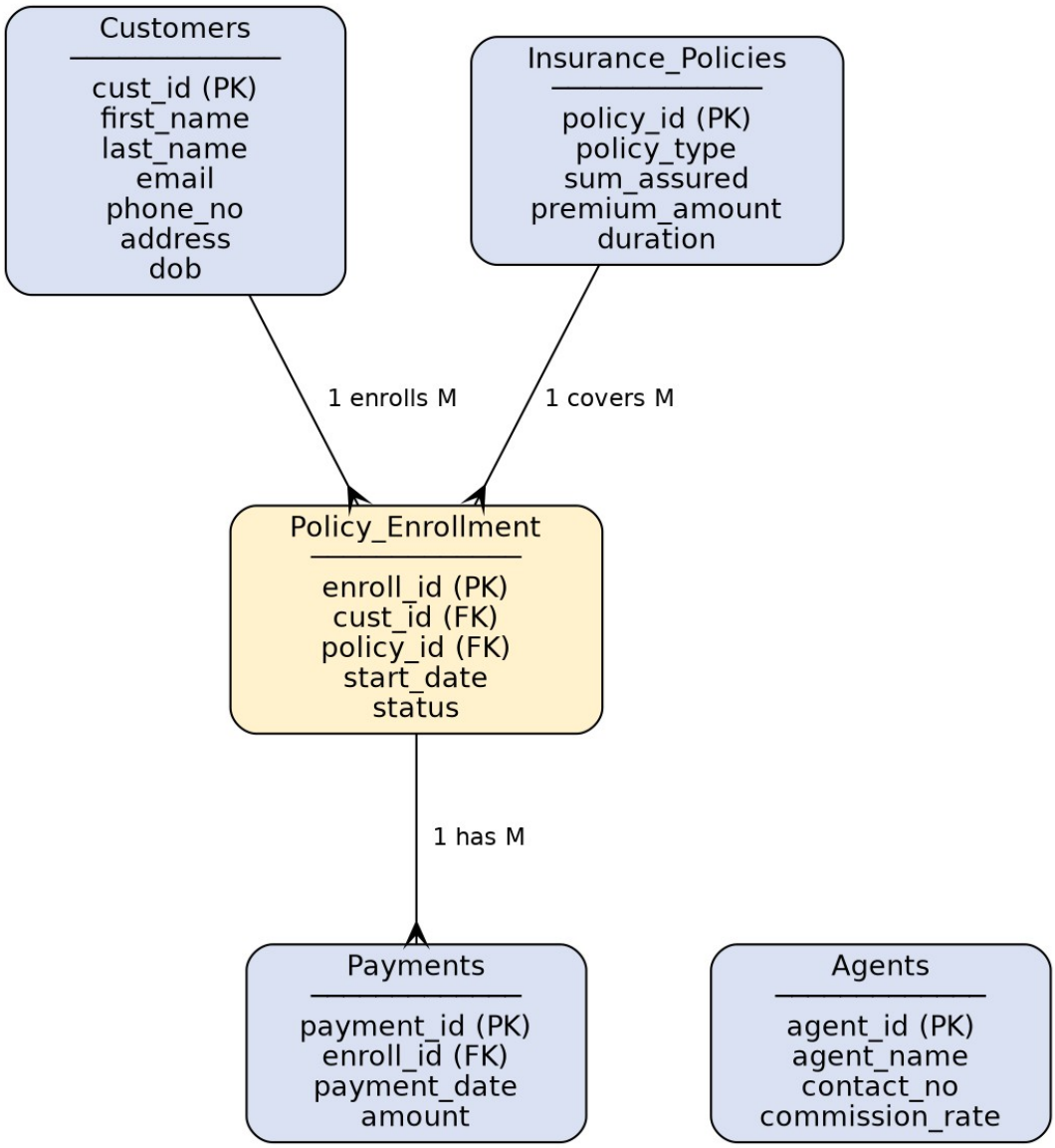

Figure 8.1: Entity Relationship (ER) Diagram

8.2 Use Case Diagram

The Use Case Diagram depicts the interactions between system actors and the functional operations supported by the IPMS. The primary actors are the Administrator and the Agent. The Administrator can register customers, create and manage policies, enrol customers, record payments, manage agent information, generate reports, and manage user access. The Agent has limited access restricted to viewing policy details and enrolment status.

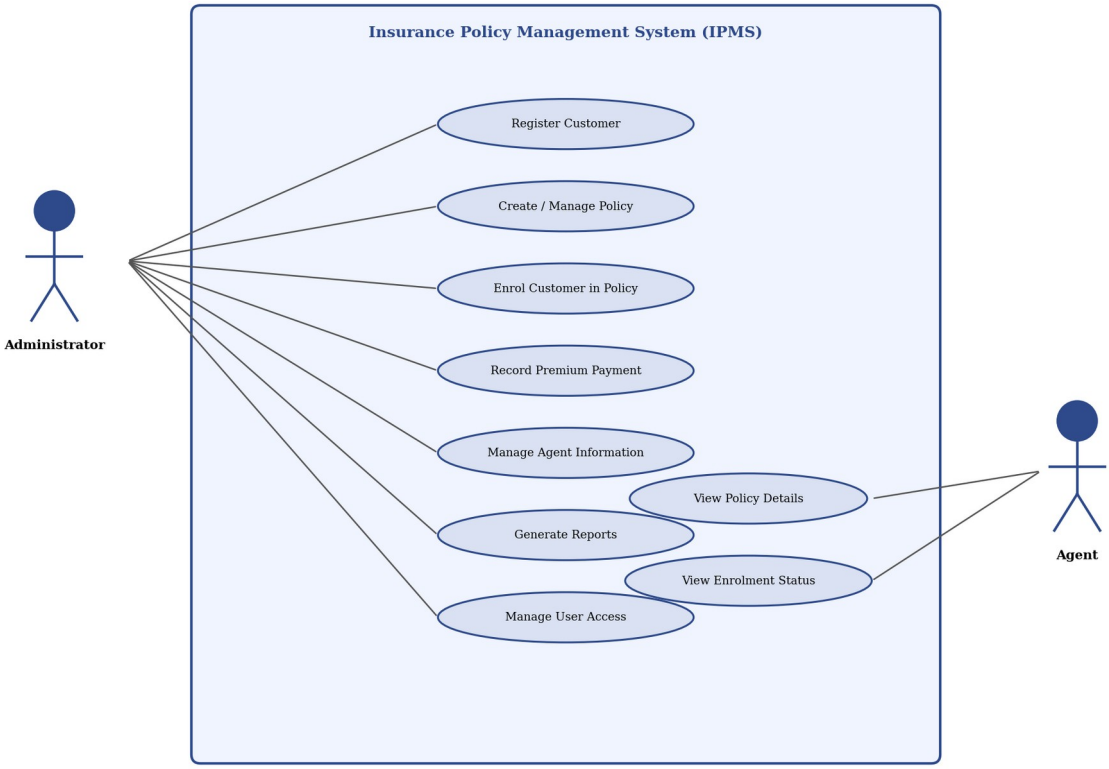

Figure 8.2: Use Case Diagram

8.3 Activity Diagram

The Activity Diagram describes the sequential workflow of key database operations within the IPMS. The administrator logs in, registers a new customer, creates an insurance policy, enrols the customer, records premium payments, and commits the transaction. Decision nodes handle cases where a customer does not require a policy or where a payment fails, triggering a ROLLBACK. Successful transactions are committed and reports are generated as required.

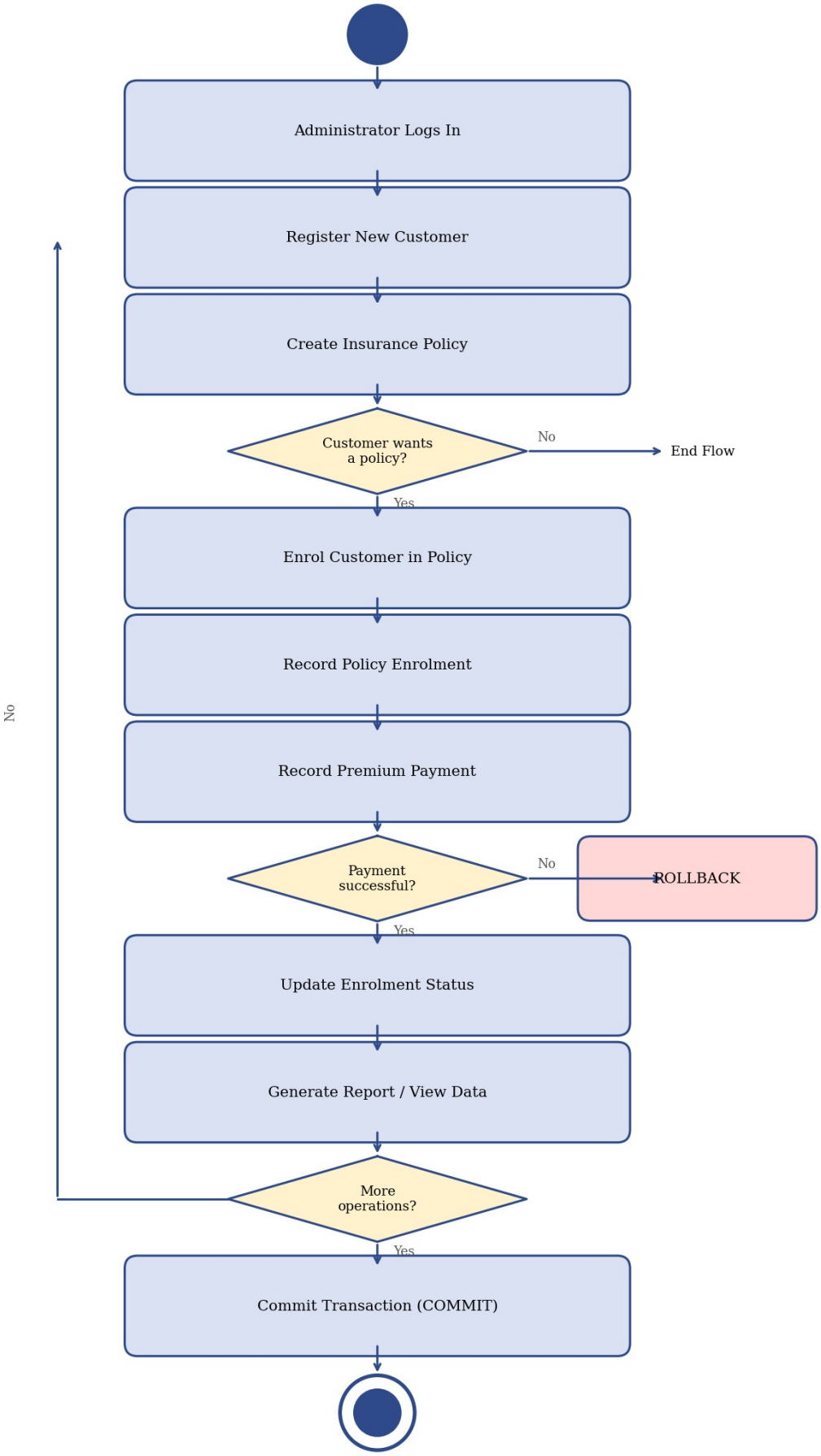
Administrator Logs In
Register New Customer
Create Insurance Policy
Customer wants a policy?
No
End Flow
Yes
Enrol Customer in Policy
Record Policy Enrolment
Record Premium Payment
Payment successful?
No
ROLLBACK
Yes
Update Enrolment Status
Generate Report / View Data
More operations?
No
Yes
Commit Transaction (COMMIT)

Figure 8.3: Activity Diagram

8.4 Sequence Diagram

The Sequence Diagram illustrates the flow of interactions between the user, MySQL Workbench, and the MySQL Server during a typical query execution cycle. The user submits a SQL query through MySQL Workbench, which forwards it to the MySQL Server. The server parses and validates the query, accesses the relevant database tables, retrieves the requested data, applies query logic, and returns the formatted result set back through MySQL Workbench to the user.

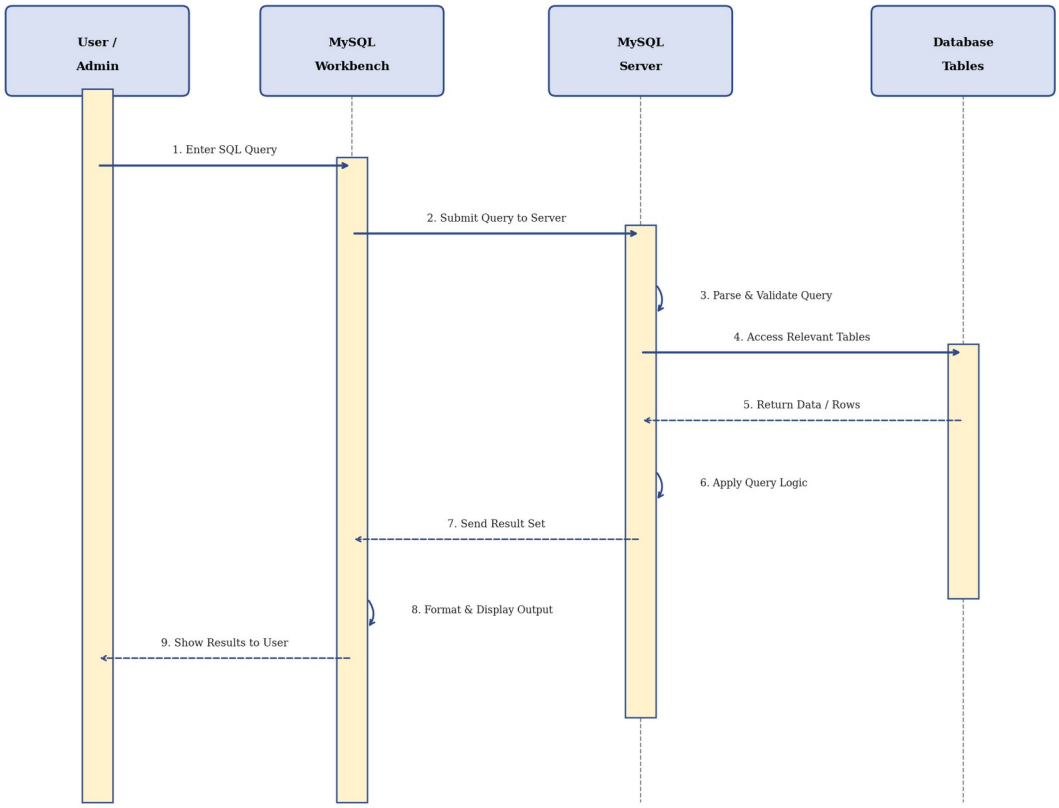

Figure 8.4: Sequence Diagram

CHAPTER 9

SQL IMPLEMENTATION

9.1 Database Creation

The following SQL command creates the database for the Insurance Policy Management System:

```
mysql> CREATE DATABASE ipms_db;
mysql> USE ipms_db;
```

Output:

```
mysql> create database ipms_db;
Query OK, 1 row affected (0.01 sec)

mysql> use ipms_db;
Database changed
mysql> |
```

9.2 Table Creation

The following Data Definition Language (DDL) commands create the tables with appropriate data types and constraints:

9.2.1 Customers Table:

```
mysql> CREATE TABLE Customers (
    ->     cust_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     first_name VARCHAR(50) NOT NULL,
    ->     last_name VARCHAR(50) NOT NULL,
    ->     email VARCHAR(100) UNIQUE NOT NULL,
    ->     phone_no VARCHAR(15) NOT NULL,
    ->     address TEXT,
    ->     dob DATE NOT NULL
    -> );
```

9.2.2 Insurance_Policies Table:

```
mysql> CREATE TABLE Insurance_Policies (
    ->     policy_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     policy_type VARCHAR(100) NOT NULL,
    ->     sum_assured DECIMAL(15,2) NOT NULL,
    ->     premium_amount DECIMAL(10,2) NOT NULL,
    ->     duration INT NOT NULL
    -> );
```

9.2.3 Policy_Enrollment Table:

```
mysql> CREATE TABLE Policy_Enrollment (
    ->     enroll_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     cust_id INT NOT NULL,
    ->     policy_id INT NOT NULL,
    ->     start_date DATE NOT NULL,
    ->     status VARCHAR(20) DEFAULT 'Active',
    ->     FOREIGN KEY (cust_id) REFERENCES Customers(cust_id),
    ->     FOREIGN KEY (policy_id) REFERENCES Insurance_Policies(policy_id)
    -> );
```

9.2.4 Payments Table:

```
mysql> CREATE TABLE Payments (
    ->     payment_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     enroll_id INT NOT NULL,
    ->     payment_date DATE NOT NULL,
    ->     amount DECIMAL(10,2) NOT NULL,
    ->     FOREIGN KEY (enroll_id) REFERENCES Policy_Enrollment(enroll_id)
    -> );
```

9.2.5 Agents Table:

```
mysql> CREATE TABLE Agents (
    ->     agent_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     agent_name VARCHAR(100) NOT NULL,
    ->     contact_no VARCHAR(15) NOT NULL,
    ->     commission_rate DECIMAL(5,2) NOT NULL
    -> );
```

9.3 Data Insertion

The following DML INSERT commands insert sample records into all database tables:

A. Insert into Customers Table:

```
mysql> INSERT INTO Customers
    -> (first_name, last_name, email, phone_no, address, dob)
    -> VALUES
    -> ('Rahul',    'Sharma',   'rahul.sharma@email.com',   '9876543210', '12 MG Road,
Mumbai',        '1990-05-15'),
    -> ('Priya',    'Mehta',    'priya.mehta@email.com',    '9123456780', '45 Andheri West,
Mumbai',   '1992-08-22'),
    -> ('Amit',     'Verma',    'amit.verma@email.com',     '9988776655', '22 Bandra,
Mumbai',         '1985-03-10'),
    -> ('Sneha',    'Patil',    'sneha.patil@email.com',    '9011223344', '67 Thane East,
Thane',      '1995-11-30'),
    -> ('Rohan',    'Joshi',    'rohan.joshi@email.com',    '8877665544', '33 Pune Road,
Pune',        '1988-07-18'),
    -> ('Anjali',   'Singh',    'anjali.singh@email.com',   '7766554433', '10 Colaba,
```

```
Mumbai',         '1993-02-14'),
    -> ('Vikram',   'Patel',    'vikram.patel@email.com',   '9654321098', '55 Dadar, Mumbai',
'1980-09-05'),
    -> ('Neha',     'Gupta',    'neha.gupta@email.com',     '9543210987', '78 Vile Parle,
Mumbai',     '1997-12-25'),
    -> ('Suresh',   'Kumar',    'suresh.kumar@email.com',   '9432109876', '90 Borivali,
Mumbai',       '1975-06-08'),
    -> ('Kavita',   'Rao',      'kavita.rao@email.com',     '9321098765', '15 Malad, Mumbai',
'1991-04-17'),
    -> ('Arjun',    'Desai',    'arjun.desai@email.com',    '8210987654', '28 Kandivali,
Mumbai',      '1986-10-22'),
    -> ('Pooja',    'Nair',     'pooja.nair@email.com',     '8109876543', '42 Goregaon,
Mumbai',       '1994-01-09'),
    -> ('Manish',   'Tiwari',   'manish.tiwari@email.com',  '7998765432', '63 Santacruz,
Mumbai',      '1983-08-14'),
    -> ('Deepika',  'Shah',     'deepika.shah@email.com',   '7887654321', '88 Khar, Mumbai',
'1990-03-28'),
    -> ('Rajesh',   'Mishra',   'rajesh.mishra@email.com',  '7776543210', '101 Juhu, Mumbai',
'1978-11-11');
Query OK, 15 rows affected (0.04 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

Display Customers Table:

```
mysql> select * from Customers;
+----------+------------+-----------+---------------------------+------------
+-----------------------+------------+
| cust_id  | first_name | last_name | email                     | phone_no   | address
| dob        |
+----------+------------+-----------+---------------------------+------------
+-----------------------+------------+
|        1 | Rahul      | Sharma    | rahul.sharma@email.com    | 9876543210 | 12 MG Road,
Mumbai    | 1990-05-15 |
|        2 | Priya      | Mehta     | priya.mehta@email.com     | 9123456780 | 45 Andheri
West...    | 1992-08-22 |
|        3 | Amit       | Verma     | amit.verma@email.com      | 9988776655 | 22 Bandra,
Mumbai     | 1985-03-10 |
|        4 | Sneha      | Patil     | sneha.patil@email.com     | 9011223344 | 67 Thane
East, Thane  | 1995-11-30 |
|        5 | Rohan      | Joshi     | rohan.joshi@email.com     | 8877665544 | 33 Pune Road,
Pune    | 1988-07-18 |
|        6 | Anjali     | Singh     | anjali.singh@email.com    | 7766554433 | 10 Colaba,
Mumbai     | 1993-02-14 |
|        7 | Vikram     | Patel     | vikram.patel@email.com    | 9654321098 | 55 Dadar,
Mumbai      | 1980-09-05 |
|        8 | Neha       | Gupta     | neha.gupta@email.com      | 9543210987 | 78 Vile
Parle, Mumbai | 1997-12-25 |
|        9 | Suresh     | Kumar     | suresh.kumar@email.com    | 9432109876 | 90 Borivali,
Mumbai   | 1975-06-08 |
|       10 | Kavita     | Rao       | kavita.rao@email.com      | 9321098765 | 15 Malad,
Mumbai      | 1991-04-17 |
|       11 | Arjun      | Desai     | arjun.desai@email.com     | 8210987654 | 28 Kandivali,
Mumbai  | 1986-10-22 |
```

```
|       12 | Pooja      | Nair      | pooja.nair@email.com       | 8109876543 | 42 Goregaon,
Mumbai   | 1994-01-09 |
|       13 | Manish     | Tiwari    | manish.tiwari@email.com    | 7998765432 | 63 Santacruz,
Mumbai  | 1983-08-14 |
|       14 | Deepika    | Shah      | deepika.shah@email.com     | 7887654321 | 88 Khar,
Mumbai       | 1990-03-28 |
|       15 | Rajesh     | Mishra    | rajesh.mishra@email.com    | 7776543210 | 101 Juhu,
Mumbai      | 1978-11-11 |
+----------+------------+-----------+----------------------------+------------
+-----------------------+------------+
15 rows in set (0.01 sec)
```

B. Insert into Insurance_Policies Table:

```
mysql> INSERT INTO Insurance_Policies
    -> (policy_type, sum_assured, premium_amount, duration)
    -> VALUES
    -> ('Life Insurance',       1000000.00, 12000.00, 20),
    -> ('Health Insurance',      500000.00,  8000.00, 10),
    -> ('Vehicle Insurance',     200000.00,  4500.00,  5),
    -> ('Term Insurance',       2000000.00, 18000.00, 25),
    -> ('Child Education Plan',  750000.00,  9500.00, 18),
    -> ('Home Insurance',        800000.00, 11000.00, 15),
    -> ('Travel Insurance',       50000.00,  1200.00,  1),
    -> ('Pension Plan',          600000.00, 10500.00, 20),
    -> ('Critical Illness Plan', 400000.00,  7200.00, 10),
    -> ('Endowment Plan',        900000.00, 14000.00, 22);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Display Insurance_Policies Table:

```
mysql> select * from Insurance_Policies;
+-----------+-----------------------+--------------+----------------+----------+
| policy_id | policy_type           | sum_assured  | premium_amount | duration |
+-----------+-----------------------+--------------+----------------+----------+
|         1 | Life Insurance        | 1000000.00   |       12000.00 |       20 |
|         2 | Health Insurance      |  500000.00   |        8000.00 |       10 |
|         3 | Vehicle Insurance     |  200000.00   |        4500.00 |        5 |
|         4 | Term Insurance        | 2000000.00   |       18000.00 |       25 |
|         5 | Child Education Plan  |  750000.00   |        9500.00 |       18 |
|         6 | Home Insurance        |  800000.00   |       11000.00 |       15 |
|         7 | Travel Insurance      |   50000.00   |        1200.00 |        1 |
|         8 | Pension Plan          |  600000.00   |       10500.00 |       20 |
|         9 | Critical Illness Plan |  400000.00   |        7200.00 |       10 |
|        10 | Endowment Plan        |  900000.00   |       14000.00 |       22 |
+-----------+-----------------------+--------------+----------------+----------+
10 rows in set (0.01 sec)
```

C. Insert into Policy_Enrollment Table:

```
mysql> INSERT INTO Policy_Enrollment
    -> (cust_id, policy_id, start_date, status)
    -> VALUES
    -> (1,  1,  '2024-01-01', 'Active'),
    -> (2,  2,  '2024-01-15', 'Active'),
    -> (3,  3,  '2024-02-01', 'Active'),
    -> (4,  4,  '2024-02-10', 'Active'),
    -> (5,  5,  '2024-03-01', 'Active'),
    -> (6,  1,  '2024-03-15', 'Active'),
    -> (7,  6,  '2024-04-01', 'Lapsed'),
    -> (8,  7,  '2024-04-10', 'Active'),
    -> (9,  8,  '2024-05-01', 'Active'),
    -> (10, 9,  '2024-05-15', 'Active'),
    -> (11, 10, '2024-06-01', 'Active'),
    -> (12, 2,  '2024-06-10', 'Lapsed'),
    -> (13, 3,  '2024-07-01', 'Active'),
    -> (14, 4,  '2024-07-15', 'Active'),
    -> (15, 5,  '2024-08-01', 'Active');
Query OK, 15 rows affected (0.03 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

Display Policy_Enrollment Table:

```
mysql> select * from Policy_Enrollment;
+-----------+---------+-----------+------------+--------+
| enroll_id | cust_id | policy_id | start_date | status |
+-----------+---------+-----------+------------+--------+
|         1 |       1 |         1 | 2024-01-01 | Active |
|         2 |       2 |         2 | 2024-01-15 | Active |
|         3 |       3 |         3 | 2024-02-01 | Active |
|         4 |       4 |         4 | 2024-02-10 | Active |
|         5 |       5 |         5 | 2024-03-01 | Active |
|         6 |       6 |         1 | 2024-03-15 | Active |
|         7 |       7 |         6 | 2024-04-01 | Lapsed |
|         8 |       8 |         7 | 2024-04-10 | Active |
|         9 |       9 |         8 | 2024-05-01 | Active |
|        10 |      10 |         9 | 2024-05-15 | Active |
|        11 |      11 |        10 | 2024-06-01 | Active |
|        12 |      12 |         2 | 2024-06-10 | Lapsed |
|        13 |      13 |         3 | 2024-07-01 | Active |
|        14 |      14 |         4 | 2024-07-15 | Active |
|        15 |      15 |         5 | 2024-08-01 | Active |
+-----------+---------+-----------+------------+--------+
15 rows in set (0.01 sec)
```

D. Insert into Payments Table:

```
mysql> INSERT INTO Payments
    -> (enroll_id, payment_date, amount)
    -> VALUES
    -> (1,  '2024-01-05', 12000.00),
    -> (2,  '2024-01-20', 8000.00),
    -> (3,  '2024-02-05', 4500.00),
    -> (4,  '2024-02-15', 18000.00),
    -> (5,  '2024-03-05', 9500.00),
    -> (6,  '2024-03-20', 12000.00),
    -> (9,  '2024-05-05', 10500.00),
    -> (10, '2024-05-20', 7200.00),
    -> (11, '2024-06-05', 14000.00),
    -> (13, '2024-07-05', 4500.00);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Display Payments Table:

```
mysql> select * from Payments;
+------------+-----------+--------------+-----------+
| payment_id | enroll_id | payment_date | amount    |
+------------+-----------+--------------+-----------+
|          1 |         1 | 2024-01-05   |  12000.00 |
|          2 |         2 | 2024-01-20   |   8000.00 |
|          3 |         3 | 2024-02-05   |   4500.00 |
|          4 |         4 | 2024-02-15   |  18000.00 |
|          5 |         5 | 2024-03-05   |   9500.00 |
|          6 |         6 | 2024-03-20   |  12000.00 |
|          7 |         9 | 2024-05-05   |  10500.00 |
|          8 |        10 | 2024-05-20   |   7200.00 |
|          9 |        11 | 2024-06-05   |  14000.00 |
|         10 |        13 | 2024-07-05   |   4500.00 |
+------------+-----------+--------------+-----------+
10 rows in set (0.01 sec)
```

E. Insert into Agents Table:

```
mysql> INSERT INTO Agents
    -> (agent_name, contact_no, commission_rate)
    -> VALUES
    -> ('Priya Mehta',    '9123456789', 5.50),
    -> ('Amit Kapoor',    '9234567890', 6.00),
    -> ('Sunita Rathi',   '9345678901', 4.75),
    -> ('Deepak Jain',    '9456789012', 7.25),
    -> ('Kavitha Nair',   '9567890123', 5.00),
    -> ('Manoj Pillai',   '9678901234', 6.50),
```

```
    -> ('Rekha Gupta',    '9789012345', 4.50),
    -> ('Sanjay Yadav',   '9890123456', 7.00),
    -> ('Lalitha Bose',   '9901234567', 5.25),
    -> ('Vimal Thakur',   '9012345678', 6.75);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Display Agents Table:

```
mysql> select * from Agents;
+----------+---------------+------------+-----------------+
| agent_id | agent_name    | contact_no | commission_rate |
+----------+---------------+------------+-----------------+
|        1 | Priya Mehta   | 9123456789 |            5.50 |
|        2 | Amit Kapoor   | 9234567890 |            6.00 |
|        3 | Sunita Rathi  | 9345678901 |            4.75 |
|        4 | Deepak Jain   | 9456789012 |            7.25 |
|        5 | Kavitha Nair  | 9567890123 |            5.00 |
|        6 | Manoj Pillai  | 9678901234 |            6.50 |
|        7 | Rekha Gupta   | 9789012345 |            4.50 |
|        8 | Sanjay Yadav  | 9890123456 |            7.00 |
|        9 | Lalitha Bose  | 9901234567 |            5.25 |
|       10 | Vimal Thakur  | 9012345678 |            6.75 |
+----------+---------------+------------+-----------------+
10 rows in set (0.01 sec)
```

9.4 Data Retrieval

Retrieve all active policy enrolments with customer and policy names (INNER JOIN):

```
mysql> SELECT C.first_name, C.last_name, IP.policy_type,
    ->        PE.start_date, PE.status
    -> FROM Policy_Enrollment PE
    -> INNER JOIN Customers C ON PE.cust_id = C.cust_id
    -> INNER JOIN Insurance_Policies IP ON PE.policy_id = IP.policy_id
    -> WHERE PE.status = 'Active';
+------------+-----------+-----------------------+------------+--------+
| first_name | last_name | policy_type           | start_date | status |
+------------+-----------+-----------------------+------------+--------+
| Rahul      | Sharma    | Life Insurance        | 2024-01-01 | Active |
| Priya      | Mehta     | Health Insurance      | 2024-01-15 | Active |
| Amit       | Verma     | Vehicle Insurance     | 2024-02-01 | Active |
| Sneha      | Patil     | Term Insurance        | 2024-02-10 | Active |
| Rohan      | Joshi     | Child Education Plan  | 2024-03-01 | Active |
| Anjali     | Singh     | Life Insurance        | 2024-03-15 | Active |
| Neha       | Gupta     | Travel Insurance      | 2024-04-10 | Active |
| Suresh     | Kumar     | Pension Plan          | 2024-05-01 | Active |
| Kavita     | Rao       | Critical Illness Plan | 2024-05-15 | Active |
```

```
| Arjun      | Desai     | Endowment Plan         | 2024-06-01 | Active |
| Manish     | Tiwari    | Vehicle Insurance      | 2024-07-01 | Active |
| Deepika    | Shah      | Term Insurance         | 2024-07-15 | Active |
| Rajesh     | Mishra    | Child Education Plan   | 2024-08-01 | Active |
+------------+-----------+------------------------+------------+--------+
13 rows in set (0.02 sec)
```

9.5 Advanced Queries

Aggregate Query — Total premium payments collected per policy type (GROUP BY + HAVING):

```
mysql> SELECT IP.policy_type, SUM(P.amount) AS total_collected
    -> FROM Payments P
    -> INNER JOIN Policy_Enrollment PE ON P.enroll_id = PE.enroll_id
    -> INNER JOIN Insurance_Policies IP ON PE.policy_id = IP.policy_id
    -> GROUP BY IP.policy_type
    -> HAVING SUM(P.amount) > 5000;
+------------------------+-----------------+
| policy_type            | total_collected |
+------------------------+-----------------+
| Life Insurance         |       24000.00  |
| Health Insurance       |        8000.00  |
| Term Insurance         |       18000.00  |
| Child Education Plan   |        9500.00  |
| Pension Plan           |       10500.00  |
| Critical Illness Plan  |        7200.00  |
| Endowment Plan         |       14000.00  |
| Vehicle Insurance      |        9000.00  |
+------------------------+-----------------+
8 rows in set (0.02 sec)
```

Subquery — Customers who paid more than the average payment amount:

```
mysql> SELECT DISTINCT C.first_name, C.last_name
    -> FROM Customers C
    -> WHERE C.cust_id IN (
    ->     SELECT PE.cust_id FROM Policy_Enrollment PE
    ->     INNER JOIN Payments P ON PE.enroll_id = P.enroll_id
    ->     WHERE P.amount > (SELECT AVG(amount) FROM Payments)
    -> );
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Rahul      | Sharma    |
| Anjali     | Singh     |
| Sneha      | Patil     |
| Suresh     | Kumar     |
| Arjun      | Desai     |
```

```
+------------+-----------+
5 rows in set (0.02 sec)
```

CREATE VIEW — Simplified access to active enrolment information:

```
mysql> CREATE VIEW Active_Enrollments AS
    -> SELECT C.first_name, C.last_name, IP.policy_type, PE.start_date
    -> FROM Policy_Enrollment PE
    -> INNER JOIN Customers C ON PE.cust_id = C.cust_id
    -> INNER JOIN Insurance_Policies IP ON PE.policy_id = IP.policy_id
    -> WHERE PE.status = 'Active';
Query OK, 0 rows affected (0.05 sec)
```

VIEW OUTPUT:

```
mysql> select * from Active_Enrollments;
+------------+-----------+-----------------------+------------+
| first_name | last_name | policy_type           | start_date |
+------------+-----------+-----------------------+------------+
| Rahul      | Sharma    | Life Insurance        | 2024-01-01 |
| Priya      | Mehta     | Health Insurance      | 2024-01-15 |
| Amit       | Verma     | Vehicle Insurance     | 2024-02-01 |
| Sneha      | Patil     | Term Insurance        | 2024-02-10 |
| Rohan      | Joshi     | Child Education Plan  | 2024-03-01 |
| Anjali     | Singh     | Life Insurance        | 2024-03-15 |
| Neha       | Gupta     | Travel Insurance      | 2024-04-10 |
| Suresh     | Kumar     | Pension Plan          | 2024-05-01 |
| Kavita     | Rao       | Critical Illness Plan | 2024-05-15 |
| Arjun      | Desai     | Endowment Plan        | 2024-06-01 |
| Manish     | Tiwari    | Vehicle Insurance     | 2024-07-01 |
| Deepika    | Shah      | Term Insurance        | 2024-07-15 |
| Rajesh     | Mishra    | Child Education Plan  | 2024-08-01 |
+------------+-----------+-----------------------+------------+
13 rows in set (0.01 sec)
```

TCL Commands — Transaction Management (COMMIT / ROLLBACK):

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO Payments (enroll_id, payment_date, amount)
    -> VALUES (1, '2024-02-05', 12000.00);
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE Policy_Enrollment SET status = 'Active'
    -> WHERE enroll_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.006 sec)
```

Rollback:

```
mysql>
mysql> ROLLBACK;
Query OK, 0 rows affected (0.006 sec)
```

DCL Commands — Access Control (GRANT / REVOKE):

```
mysql> GRANT SELECT ON ipms_db.Active_Enrollments TO 'agent_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> REVOKE DELETE ON ipms_db.Customers FROM 'staff_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```

CHAPTER 10

SYSTEM TESTING AND RESULT

System testing ensures that all SQL queries function correctly, data integrity constraints are enforced, and the expected output is produced for given inputs. The IPMS was tested systematically for query correctness, constraint validation, and output verification.

10.1 Query Correctness Testing

Each SQL query was executed individually in MySQL Workbench and the output was verified against expected results. DDL commands were tested to confirm correct table structures, and DML commands were tested to verify accurate data operations.

10.2 Constraint Validation

- Inserting a duplicate email address returned an error due to the UNIQUE constraint.
- Inserting a Policy_Enrollment record with a non-existent cust_id returned a foreign key violation error.
- Inserting a NULL value in a NOT NULL field produced an appropriate error message.
- Attempting to delete a customer with existing enrolments was correctly rejected by the referential integrity constraint.

10.3 Output Verification

The results of SELECT queries and JOIN operations were verified against inserted sample data. Aggregate functions were manually cross-checked and the Active_Enrollments view correctly displayed only 'Active' status enrolments.

Table 10.1: Test Case Summary

10.3 Sample Output (Result Tables)

The following outputs display actual MySQL Workbench execution results for all SQL operations performed in the Insurance Policy Management System.

10.3.1 Database Creation and USE Database

```
mysql> CREATE DATABASE ipms_db;
Query OK, 1 row affected (0.01 sec)

mysql> USE ipms_db;
Database changed
mysql> |
```

10.3.2 Table Creation

A. Customers Table:

```
mysql> CREATE TABLE Customers (
    ->     cust_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     first_name VARCHAR(50) NOT NULL,
    ->     last_name VARCHAR(50) NOT NULL,
    ->     email VARCHAR(100) UNIQUE NOT NULL,
    ->     phone_no VARCHAR(15) NOT NULL,
    ->     address TEXT,
    ->     dob DATE NOT NULL
    -> );
Query OK, 0 rows affected (0.06 sec)
```

B. Insurance_Policies Table:

```
mysql> CREATE TABLE Insurance_Policies (
    ->     policy_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     policy_type VARCHAR(100) NOT NULL,
    ->     sum_assured DECIMAL(15,2) NOT NULL,
    ->     premium_amount DECIMAL(10,2) NOT NULL,
    ->     duration INT NOT NULL
    -> );
Query OK, 0 rows affected (0.05 sec)
```

C. Policy_Enrollment Table:

```
mysql> CREATE TABLE Policy_Enrollment (
    ->     enroll_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     cust_id INT NOT NULL,
    ->     policy_id INT NOT NULL,
    ->     start_date DATE NOT NULL,
    ->     status VARCHAR(20) DEFAULT 'Active',
    ->     FOREIGN KEY (cust_id) REFERENCES Customers(cust_id),
    ->     FOREIGN KEY (policy_id) REFERENCES Insurance_Policies(policy_id)
```

```
    -> );
Query OK, 0 rows affected (0.07 sec)
```

D. Payments Table:

```
mysql> CREATE TABLE Payments (
    ->     payment_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     enroll_id INT NOT NULL,
    ->     payment_date DATE NOT NULL,
    ->     amount DECIMAL(10,2) NOT NULL,
    ->     FOREIGN KEY (enroll_id) REFERENCES Policy_Enrollment(enroll_id)
    -> );
Query OK, 0 rows affected (0.05 sec)
```

E. Agents Table:

```
mysql> CREATE TABLE Agents (
    ->     agent_id INT AUTO_INCREMENT PRIMARY KEY,
    ->     agent_name VARCHAR(100) NOT NULL,
    ->     contact_no VARCHAR(15) NOT NULL,
    ->     commission_rate DECIMAL(5,2) NOT NULL
    -> );
Query OK, 0 rows affected (0.04 sec)
```

10.3.3 Table Value Insertion and Display

A. Insert into Customers Table:

```
mysql> INSERT INTO Customers (first_name, last_name, email, phone_no, address, dob)
    -> VALUES
    -> ('Rahul','Sharma','rahul.sharma@email.com','9876543210','12 MG Road, Mumbai','1990-05-15'),
    -> ('Priya','Mehta','priya.mehta@email.com','9123456780','45 Andheri West, Mumbai','1992-08-22'),
    -> ('Amit','Verma','amit.verma@email.com','9988776655','22 Bandra, Mumbai','1985-03-10'),
    -> ('Sneha','Patil','sneha.patil@email.com','9011223344','67 Thane East, Thane','1995-11-30'),
    -> ('Rohan','Joshi','rohan.joshi@email.com','8877665544','33 Pune Road, Pune','1988-07-18'),
    -> ('Anjali','Singh','anjali.singh@email.com','7766554433','10 Colaba, Mumbai','1993-02-14'),
    -> ('Vikram','Patel','vikram.patel@email.com','9654321098','55 Dadar, Mumbai','1980-09-05'),
    -> ('Neha','Gupta','neha.gupta@email.com','9543210987','78 Vile Parle, Mumbai','1997-12-25'),
    -> ('Suresh','Kumar','suresh.kumar@email.com','9432109876','90 Borivali, Mumbai','1975-06-08'),
    -> ('Kavita','Rao','kavita.rao@email.com','9321098765','15 Malad, Mumbai','1991-04-17'),
    -> ('Arjun','Desai','arjun.desai@email.com','8210987654','28 Kandivali, Mumbai','1986-10-22'),
    -> ('Pooja','Nair','pooja.nair@email.com','8109876543','42 Goregaon, Mumbai','1994-01-09'),
    -> ('Manish','Tiwari','manish.tiwari@email.com','7998765432','63 Santacruz, Mumbai','1983-08-14'),
    -> ('Deepika','Shah','deepika.shah@email.com','7887654321','88 Khar, Mumbai','1990-03-28'),
    -> ('Rajesh','Mishra','rajesh.mishra@email.com','7776543210','101 Juhu, Mumbai','1978-11-11');
Query OK, 15 rows affected (0.04 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

Display Customers Table:

```
mysql> SELECT * FROM Customers;
+---------+------------+-----------+----------------------------+------------+-----------------------+-----------+
| cust_id | first_name | last_name | email                      | phone_no   | address               | dob       |
+---------+------------+-----------+----------------------------+------------+-----------------------+-----------+
|       1 | Rahul      | Sharma    | rahul.sharma@email.com     | 9876543210 | 12 MG Road, Mumbai    | 1990-05-15 |
|       2 | Priya      | Mehta     | priya.mehta@email.com      | 9123456780 | 45 Andheri West, Mumbai| 1992-08-22 |
|       3 | Amit       | Verma     | amit.verma@email.com       | 9988776655 | 22 Bandra, Mumbai     | 1985-03-10 |
|       4 | Sneha      | Patil     | sneha.patil@email.com      | 9011223344 | 67 Thane East, Thane  | 1995-11-30 |
|       5 | Rohan      | Joshi     | rohan.joshi@email.com      | 8877665544 | 33 Pune Road, Pune    | 1988-07-18 |
|       6 | Anjali     | Singh     | anjali.singh@email.com     | 7766554433 | 10 Colaba, Mumbai     | 1993-02-14 |
|       7 | Vikram     | Patel     | vikram.patel@email.com     | 9654321098 | 55 Dadar, Mumbai      | 1980-09-05 |
|       8 | Neha       | Gupta     | neha.gupta@email.com       | 9543210987 | 78 Vile Parle, Mumbai | 1997-12-25 |
|       9 | Suresh     | Kumar     | suresh.kumar@email.com     | 9432109876 | 90 Borivali, Mumbai   | 1975-06-08 |
|      10 | Kavita     | Rao       | kavita.rao@email.com       | 9321098765 | 15 Malad, Mumbai      | 1991-04-17 |
|      11 | Arjun      | Desai     | arjun.desai@email.com      | 8210987654 | 28 Kandivali, Mumbai  | 1986-10-22 |
|      12 | Pooja      | Nair      | pooja.nair@email.com       | 8109876543 | 42 Goregaon, Mumbai   | 1994-01-09 |
|      13 | Manish     | Tiwari    | manish.tiwari@email.com    | 7998765432 | 63 Santacruz, Mumbai  | 1983-08-14 |
|      14 | Deepika    | Shah      | deepika.shah@email.com     | 7887654321 | 88 Khar, Mumbai       | 1990-03-28 |
|      15 | Rajesh     | Mishra    | rajesh.mishra@email.com    | 7776543210 | 101 Juhu, Mumbai      | 1978-11-11 |
+---------+------------+-----------+----------------------------+------------+-----------------------+-----------+
15 rows in set (0.01 sec)
```

B. Insert into Insurance_Policies Table:

```
mysql> INSERT INTO Insurance_Policies (policy_type, sum_assured, premium_amount, duration)
    -> VALUES
    -> ('Life Insurance',      1000000.00, 12000.00, 20),
    -> ('Health Insurance',     500000.00,  8000.00, 10),
    -> ('Vehicle Insurance',    200000.00,  4500.00,  5),
    -> ('Term Insurance',      2000000.00, 18000.00, 25),
    -> ('Child Education Plan', 750000.00,  9500.00, 18),
    -> ('Home Insurance',       800000.00, 11000.00, 15),
    -> ('Travel Insurance',      50000.00,  1200.00,  1),
    -> ('Pension Plan',         600000.00, 10500.00, 20),
    -> ('Critical Illness Plan',400000.00,  7200.00, 10),
    -> ('Endowment Plan',       900000.00, 14000.00, 22);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Display Insurance_Policies Table:

```
mysql> SELECT * FROM Insurance_Policies;
+-----------+-----------------------+-------------+----------------+----------+
| policy_id | policy_type           | sum_assured | premium_amount | duration |
+-----------+-----------------------+-------------+----------------+----------+
|         1 | Life Insurance        | 1000000.00  |       12000.00 |       20 |
|         2 | Health Insurance      |  500000.00  |        8000.00 |       10 |
|         3 | Vehicle Insurance     |  200000.00  |        4500.00 |        5 |
|         4 | Term Insurance        | 2000000.00  |       18000.00 |       25 |
|         5 | Child Education Plan  |  750000.00  |        9500.00 |       18 |
|         6 | Home Insurance        |  800000.00  |       11000.00 |       15 |
|         7 | Travel Insurance      |   50000.00  |        1200.00 |        1 |
|         8 | Pension Plan          |  600000.00  |       10500.00 |       20 |
|         9 | Critical Illness Plan |  400000.00  |        7200.00 |       10 |
|        10 | Endowment Plan        |  900000.00  |       14000.00 |       22 |
+-----------+-----------------------+-------------+----------------+----------+
10 rows in set (0.01 sec)
```

C. Insert into Policy_Enrollment Table:

```
mysql> INSERT INTO Policy_Enrollment (cust_id, policy_id, start_date, status)
    -> VALUES
    -> (1, 1,'2024-01-01','Active'), (2, 2,'2024-01-15','Active'),
    -> (3, 3,'2024-02-01','Active'), (4, 4,'2024-02-10','Active'),
    -> (5, 5,'2024-03-01','Active'), (6, 1,'2024-03-15','Active'),
    -> (7, 6,'2024-04-01','Lapsed'), (8, 7,'2024-04-10','Active'),
    -> (9, 8,'2024-05-01','Active'), (10,9,'2024-05-15','Active'),
    -> (11,10,'2024-06-01','Active'),(12,2,'2024-06-10','Lapsed'),
    -> (13,3,'2024-07-01','Active'), (14,4,'2024-07-15','Active'),
    -> (15,5,'2024-08-01','Active');
Query OK, 15 rows affected (0.03 sec)
Records: 15  Duplicates: 0  Warnings: 0
```

Display Policy_Enrollment Table:

```
mysql> SELECT * FROM Policy_Enrollment;
+-----------+---------+-----------+------------+--------+
| enroll_id | cust_id | policy_id | start_date | status |
+-----------+---------+-----------+------------+--------+
|         1 |       1 |         1 | 2024-01-01 | Active |
|         2 |       2 |         2 | 2024-01-15 | Active |
|         3 |       3 |         3 | 2024-02-01 | Active |
|         4 |       4 |         4 | 2024-02-10 | Active |
|         5 |       5 |         5 | 2024-03-01 | Active |
|         6 |       6 |         1 | 2024-03-15 | Active |
|         7 |       7 |         6 | 2024-04-01 | Lapsed |
|         8 |       8 |         7 | 2024-04-10 | Active |
|         9 |       9 |         8 | 2024-05-01 | Active |
|        10 |      10 |         9 | 2024-05-15 | Active |
|        11 |      11 |        10 | 2024-06-01 | Active |
|        12 |      12 |         2 | 2024-06-10 | Lapsed |
|        13 |      13 |         3 | 2024-07-01 | Active |
|        14 |      14 |         4 | 2024-07-15 | Active |
|        15 |      15 |         5 | 2024-08-01 | Active |
+-----------+---------+-----------+------------+--------+
15 rows in set (0.01 sec)
```

D. Insert into Payments Table:

```
mysql> INSERT INTO Payments (enroll_id, payment_date, amount)
    -> VALUES
    -> (1, '2024-01-05',12000.00), (2, '2024-01-20', 8000.00),
    -> (3, '2024-02-05', 4500.00), (4, '2024-02-15',18000.00),
    -> (5, '2024-03-05', 9500.00), (6, '2024-03-20',12000.00),
    -> (9, '2024-05-05',10500.00), (10,'2024-05-20', 7200.00),
    -> (11,'2024-06-05',14000.00), (13,'2024-07-05', 4500.00);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Display Payments Table:

```
mysql> SELECT * FROM Payments;
+------------+-----------+--------------+-----------+
| payment_id | enroll_id | payment_date | amount    |
+------------+-----------+--------------+-----------+
|          1 |         1 | 2024-01-05   |  12000.00 |
|          2 |         2 | 2024-01-20   |   8000.00 |
|          3 |         3 | 2024-02-05   |   4500.00 |
|          4 |         4 | 2024-02-15   |  18000.00 |
|          5 |         5 | 2024-03-05   |   9500.00 |
|          6 |         6 | 2024-03-20   |  12000.00 |
|          7 |         9 | 2024-05-05   |  10500.00 |
|          8 |        10 | 2024-05-20   |   7200.00 |
|          9 |        11 | 2024-06-05   |  14000.00 |
|         10 |        13 | 2024-07-05   |   4500.00 |
+------------+-----------+--------------+-----------+
10 rows in set (0.01 sec)
```

E. Insert into Agents Table:

```
mysql> INSERT INTO Agents (agent_name, contact_no, commission_rate)
    -> VALUES
    -> ('Priya Mehta',  '9123456789', 5.50), ('Amit Kapoor', '9234567890', 6.00),
    -> ('Sunita Rathi', '9345678901', 4.75), ('Deepak Jain', '9456789012', 7.25),
    -> ('Kavitha Nair', '9567890123', 5.00), ('Manoj Pillai','9678901234', 6.50),
    -> ('Rekha Gupta',  '9789012345', 4.50), ('Sanjay Yadav','9890123456', 7.00),
    -> ('Lalitha Bose', '9901234567', 5.25), ('Vimal Thakur','9012345678', 6.75);
Query OK, 10 rows affected (0.02 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Display Agents Table:

```
mysql> SELECT * FROM Agents;
+----------+---------------+------------+-----------------+
| agent_id | agent_name    | contact_no | commission_rate |
+----------+---------------+------------+-----------------+
|        1 | Priya Mehta   | 9123456789 |            5.50 |
```

```
|         2 | Amit Kapoor    | 9234567890 |            6.00 |
|         3 | Sunita Rathi   | 9345678901 |            4.75 |
|         4 | Deepak Jain    | 9456789012 |            7.25 |
|         5 | Kavitha Nair   | 9567890123 |            5.00 |
|         6 | Manoj Pillai   | 9678901234 |            6.50 |
|         7 | Rekha Gupta    | 9789012345 |            4.50 |
|         8 | Sanjay Yadav   | 9890123456 |            7.00 |
|         9 | Lalitha Bose   | 9901234567 |            5.25 |
|        10 | Vimal Thakur   | 9012345678 |            6.75 |
+----------+---------------+------------+-----------------+
10 rows in set (0.01 sec)
```

10.3.4 Constraint Validation Output

UNIQUE Constraint Violation (Duplicate Email):

```
mysql> INSERT INTO Customers (first_name, last_name, email, phone_no, dob)
    -> VALUES ('Test','User','rahul.sharma@email.com','9000000001','2000-01-01');
ERROR 1062 (23000): Duplicate entry 'rahul.sharma@email.com' for key 'email'
```

FOREIGN KEY Constraint Violation (Invalid cust_id):

```
mysql> INSERT INTO Policy_Enrollment (cust_id, policy_id, start_date, status)
    -> VALUES (999, 1, '2024-09-01', 'Active');
ERROR 1452 (23000): Cannot add or update a child row: a foreign key constraint fails
(`ipms_db`.`Policy_Enrollment`, CONSTRAINT `Policy_Enrollment_ibfk_1`
 FOREIGN KEY (`cust_id`) REFERENCES `Customers` (`cust_id`))
```

NOT NULL Constraint Violation:

```
mysql> INSERT INTO Customers (first_name, last_name, email, phone_no, dob)
    -> VALUES (NULL, 'Test', 'test@email.com', '9000000002', '2000-01-01');
ERROR 1048 (23000): Column 'first_name' cannot be null
```

Referential Integrity — Delete Rejected:

```
mysql> DELETE FROM Customers WHERE cust_id = 1;
ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails
(`ipms_db`.`Policy_Enrollment`, CONSTRAINT `Policy_Enrollment_ibfk_1`
 FOREIGN KEY (`cust_id`) REFERENCES `Customers` (`cust_id`))
```

10.3.5 Advanced Query Output

INNER JOIN — Active Enrolments with Customer and Policy Details:

```
mysql> SELECT C.first_name, C.last_name, IP.policy_type, PE.start_date, PE.status
    -> FROM Policy_Enrollment PE
    -> INNER JOIN Customers C ON PE.cust_id = C.cust_id
    -> INNER JOIN Insurance_Policies IP ON PE.policy_id = IP.policy_id
    -> WHERE PE.status = 'Active';
+------------+-----------+-----------------------+------------+--------+
| first_name | last_name | policy_type           | start_date | status |
+------------+-----------+-----------------------+------------+--------+
| Rahul      | Sharma    | Life Insurance        | 2024-01-01 | Active |
| Priya      | Mehta     | Health Insurance      | 2024-01-15 | Active |
| Amit       | Verma     | Vehicle Insurance     | 2024-02-01 | Active |
| Sneha      | Patil     | Term Insurance        | 2024-02-10 | Active |
| Rohan      | Joshi     | Child Education Plan  | 2024-03-01 | Active |
| Anjali     | Singh     | Life Insurance        | 2024-03-15 | Active |
| Neha       | Gupta     | Travel Insurance      | 2024-04-10 | Active |
| Suresh     | Kumar     | Pension Plan          | 2024-05-01 | Active |
| Kavita     | Rao       | Critical Illness Plan | 2024-05-15 | Active |
| Arjun      | Desai     | Endowment Plan        | 2024-06-01 | Active |
| Manish     | Tiwari    | Vehicle Insurance     | 2024-07-01 | Active |
| Deepika    | Shah      | Term Insurance        | 2024-07-15 | Active |
| Rajesh     | Mishra    | Child Education Plan  | 2024-08-01 | Active |
+------------+-----------+-----------------------+------------+--------+
13 rows in set (0.02 sec)
```

GROUP BY + HAVING — Total Premium Collected per Policy Type:

```
mysql> SELECT IP.policy_type, SUM(P.amount) AS total_collected
    -> FROM Payments P
    -> INNER JOIN Policy_Enrollment PE ON P.enroll_id = PE.enroll_id
    -> INNER JOIN Insurance_Policies IP ON PE.policy_id = IP.policy_id
    -> GROUP BY IP.policy_type
    -> HAVING SUM(P.amount) > 5000;
+-----------------------+----------------+
| policy_type           | total_collected |
+-----------------------+----------------+
| Life Insurance        |       24000.00  |
| Health Insurance      |        8000.00  |
| Term Insurance        |       18000.00  |
| Child Education Plan  |        9500.00  |
| Pension Plan          |       10500.00  |
| Critical Illness Plan |        7200.00  |
| Endowment Plan        |       14000.00  |
| Vehicle Insurance     |        9000.00  |
+-----------------------+----------------+
8 rows in set (0.02 sec)
```

SUBQUERY — Customers Who Paid More Than Average:

```
mysql> SELECT DISTINCT C.first_name, C.last_name
    -> FROM Customers C
    -> WHERE C.cust_id IN (
    ->     SELECT PE.cust_id FROM Policy_Enrollment PE
    ->     INNER JOIN Payments P ON PE.enroll_id = P.enroll_id
    ->     WHERE P.amount > (SELECT AVG(amount) FROM Payments)
    -> );
+------------+-----------+
| first_name | last_name |
+------------+-----------+
| Rahul      | Sharma    |
| Anjali     | Singh     |
| Sneha      | Patil     |
| Suresh     | Kumar     |
| Arjun      | Desai     |
+------------+-----------+
5 rows in set (0.02 sec)
```

10.3.6 View Creation and Output

CREATE VIEW:

```
mysql> CREATE VIEW Active_Enrollments AS
    -> SELECT C.first_name, C.last_name, IP.policy_type, PE.start_date
    -> FROM Policy_Enrollment PE
    -> INNER JOIN Customers C ON PE.cust_id = C.cust_id
    -> INNER JOIN Insurance_Policies IP ON PE.policy_id = IP.policy_id
    -> WHERE PE.status = 'Active';
Query OK, 0 rows affected (0.05 sec)
```

VIEW Output:

```
mysql> SELECT * FROM Active_Enrollments;
+------------+-----------+-----------------------+------------+
| first_name | last_name | policy_type           | start_date |
+------------+-----------+-----------------------+------------+
| Rahul      | Sharma    | Life Insurance        | 2024-01-01 |
| Priya      | Mehta     | Health Insurance      | 2024-01-15 |
| Amit       | Verma     | Vehicle Insurance     | 2024-02-01 |
| Sneha      | Patil     | Term Insurance        | 2024-02-10 |
| Rohan      | Joshi     | Child Education Plan  | 2024-03-01 |
| Anjali     | Singh     | Life Insurance        | 2024-03-15 |
| Neha       | Gupta     | Travel Insurance      | 2024-04-10 |
| Suresh     | Kumar     | Pension Plan          | 2024-05-01 |
| Kavita     | Rao       | Critical Illness Plan | 2024-05-15 |
| Arjun      | Desai     | Endowment Plan        | 2024-06-01 |
| Manish     | Tiwari    | Vehicle Insurance     | 2024-07-01 |
| Deepika    | Shah      | Term Insurance        | 2024-07-15 |
| Rajesh     | Mishra    | Child Education Plan  | 2024-08-01 |
+------------+-----------+-----------------------+------------+
13 rows in set (0.01 sec)
```

10.3.7 Transaction Control (TCL) Output

COMMIT Transaction:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql> INSERT INTO Payments (enroll_id, payment_date, amount)
    -> VALUES (1, '2024-02-05', 12000.00);
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE Policy_Enrollment SET status = 'Active' WHERE enroll_id = 1;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

```
mysql> COMMIT;
Query OK, 0 rows affected (0.006 sec)
```

ROLLBACK Transaction:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.01 sec)

mysql> UPDATE Policy_Enrollment SET status = 'Lapsed' WHERE enroll_id = 3;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> ROLLBACK;
Query OK, 0 rows affected (0.006 sec)
```

10.3.8 Data Control Language (DCL) Output

GRANT and REVOKE Commands:

```
mysql> CREATE USER 'agent_user'@'localhost' IDENTIFIED BY 'agent@123';
Query OK, 0 rows affected (0.02 sec)

mysql> GRANT SELECT ON ipms_db.Active_Enrollments TO 'agent_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> CREATE USER 'staff_user'@'localhost' IDENTIFIED BY 'staff@456';
Query OK, 0 rows affected (0.01 sec)

mysql> GRANT SELECT, INSERT, UPDATE ON ipms_db.* TO 'staff_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)

mysql> REVOKE DELETE ON ipms_db.Customers FROM 'staff_user'@'localhost';
Query OK, 0 rows affected (0.01 sec)
```

Test Case	SQL Operation	Expected Result	Status
TC01	INSERT valid customer	Record inserted successfully	PASS
TC02	INSERT duplicate email	Error: Duplicate entry	PASS
TC03	INSERT NULL in NOT NULL field	Error: Column cannot be null	PASS
TC04	INNER JOIN query	Correct joined output	PASS
TC05	SUM aggregate function	Correct total amount	PASS
TC06	View query	Only active enrolments shown	PASS
TC07	ROLLBACK transaction	Changes undone successfully	PASS

CHAPTER 11

SECURITY, BACKUP AND RECOVERY

Security and data protection are of paramount importance in any system that handles sensitive personal and financial data. The IPMS incorporates several security mechanisms to protect data from unauthorised access, accidental loss, and corruption.

11.1 Security Measures

- Role-Based Access Control (RBAC): Administrators have full access, office staff have INSERT and UPDATE rights, and agents receive SELECT privileges on relevant views only.
- Password-Protected Accounts: All MySQL user accounts are protected with strong passwords to prevent unauthorised login.
- View-Based Security: Sensitive tables are not directly accessible to lower-privileged users, who interact only with carefully designed views.

11.2 Backup Strategy

Regular database backups are essential to prevent data loss. The IPMS employs the mysqldump utility for creating logical backups.

```
mysqldump -u root -p ipms_db > ipms_backup.sql
```

It is recommended to schedule this backup at regular intervals, such as daily or weekly, depending on the frequency of data changes.

11.3 Recovery Procedure

In the event of data loss, the database can be restored from the backup file using the following command:

```
mysql -u root -p ipms_db < ipms_backup.sql
```

This command reads the SQL dump file and re-creates the database tables along with all previously stored data, restoring the system to its last backed-up state.

CHAPTER 12

FUTURE SCOPE AND CONCLUSION

12.1 Future Scope

- Web-Based Front-End Integration: A GUI developed using PHP, Python (Django/Flask), or Node.js can be integrated with the MySQL backend to provide a browser-based application accessible to non-technical staff.
- Mobile Application Support: A mobile application for Android or iOS can be developed to allow agents and customers to access policy information and payment records from smartphones.
- Stored Procedures and Triggers: Stored procedures can encapsulate complex business logic within the database, while triggers can automate actions such as sending payment reminders or updating policy status.
- Advanced Reporting and Analytics: Integration with business intelligence tools such as Tableau or Power BI can provide graphical dashboards and in-depth analytics on premium collection trends.
- Multi-User Concurrency Management: Transaction isolation levels and locking mechanisms will allow multiple simultaneous users without data conflicts.
- Cloud Deployment: Migrating the database to AWS RDS, Google Cloud SQL, or Microsoft Azure SQL will provide scalability, high availability, and disaster recovery.

12.2 Conclusion

The Insurance Policy Management System successfully demonstrates the practical application of Relational Database Management System concepts in a real-world domain. By designing a normalised MySQL database with structured tables, enforcing data integrity through constraints, and implementing SQL operations covering DDL, DML, DCL, and TCL categories, this project achieves all its stated objectives.

The project reinforced theoretical knowledge gained in the Semester IV RDBMS curriculum and provided valuable hands-on experience with MySQL and MySQL Workbench. The process of designing entities, establishing relationships, and writing

complex SQL queries including JOINs, aggregate functions, subqueries, and views has significantly enhanced understanding of database engineering principles.

The IPMS demonstrates that a well-designed relational database can eliminate the inefficiencies of manual record-keeping, ensure data accuracy, and provide a scalable foundation for a fully functional insurance management application. The knowledge and skills acquired through this project are directly applicable to professional database development and administration roles in the IT industry.

CHAPTER 13

REFERENCES

The following sources were referenced during the development and documentation of the Insurance Policy Management System:

1. MySQL 8.0 Reference Manual. Oracle Corporation. Available at: https://dev.mysql.com/doc/
2. B.Sc. IT Semester IV: Database Management Systems Textbook. University of Mumbai Prescribed Syllabus.
3. W3Schools SQL Tutorial. Available at: https://www.w3schools.com/sql/
4. TutorialsPoint MySQL Tutorial. Available at: https://www.tutorialspoint.com/mysql/
5. Elmasri, R. and Navathe, S.B. (2015). Fundamentals of Database Systems (7th ed.). Pearson Education.

CHAPTER 14

GLOSSARY

Table 14.1: Glossary of Key Terms

Term	Definition
DBMS	Database Management System – software that manages the creation, maintenance, and use of databases.
RDBMS	Relational Database Management System – a DBMS that organises data into tables with defined relationships.
SQL	Structured Query Language – the standard language used to communicate with relational databases.
MySQL	An open-source relational database management system widely used for web and enterprise applications.
Primary Key	A column or set of columns that uniquely identifies each row in a database table.
Foreign Key	A column that establishes a link between data in two tables by referencing the primary key of another table.
Normalisation	The process of organising a database to reduce redundancy and ensure logical data dependency.
DDL	Data Definition Language – SQL commands used to define database structures (CREATE, ALTER, DROP).
DML	Data Manipulation Language – SQL commands used to manipulate data (INSERT, UPDATE, DELETE, SELECT).
DCL	Data Control Language – SQL commands used to control user access (GRANT, REVOKE).
TCL	Transaction Control Language – SQL commands used to manage database transactions (COMMIT, ROLLBACK).
View	A virtual table created by a SQL query that presents data from one or more tables in a specific format.
mysqldump	A MySQL utility used to create logical backups of databases in SQL format.
3NF	Third Normal Form – a level of normalisation that eliminates transitive dependencies.